Final report: On the Implementability of Global Types

Student: Gabriele Genovese
Supervisor: Cinzia Di Giusto

May 26, 2025

Abstract

Behavioral types help define how information is exchanged in distributed systems. Multiparty Session Types (MPST) describe interactions between multiple participants using global protocols and their local versions. Ensuring correct implementation, including deadlock freedom and session conformance, is a key concern in MPST. Most research focuses on peer-to-peer communication, but real-world systems use different models like mailbox-based or causally ordered messaging. A major challenge is that a protocol working in one model may fail in another. In this work, we explore a flexible MPST framework that adapts to various communication models, including unordered, causally ordered, and bus-based messaging. Implementability is defined through quasi-synchronous (QS) local types, which ensure deadlock freedom and prevent orphan messages while maintaining compatibility with synchronous semantics. This approach extends MPST beyond peer-to-peer models, making it useful for quasi-synchronous systems, which offer better guarantees for verifying correctness.

Contents

1 Introduction **3**

2 Preliminaries **3**
2.1 Multiparty Session Types 3
2.1.1 The Type System 3
2.1.2 The Calculus' Syntax 5
2.2 Message Sequence Charts 6
2.2.1 Communication models 7

3 State of the art **9**
3.1 Realizability by Alur 9
3.1.1 Definitions 9
3.1.2 Realizability problem 10
3.2 Implementability by Stutz 12
3.2.1 Definitions 13

1 Introduction

2 Preliminaries

In this section, we define the basic knowledge to understand the main contributions of this work.

2.1 Multiparty Session Types

Multiparty Session Types (MPST) provide a formal framework to specify and verify communication protocols among multiple participants. They ensure that communication follows a predefined structure, preventing errors such as deadlocks and message loss. Below, we present the fundamental definitions of MPST. The **global specification** describes the overall communication protocol. From this, we derive the **local behaviors** of each participant through a *projection* operation. The system's **processes** form the *implementation*, defining how participants interact. With the definition of a *typing system* and the application of the *type-checking rules*, we ensure that the implementation conforms to the global specification, guaranteeing *well-formedness*.

2.1.1 The Type System

Global Types A global type G describes the overall communication protocol, specifying the sequence of message exchanges among participants. It defines the expected interactions at a high level. We give the syntax of global types following work by Honda et al. [2]. The focus is on the core message passing aspects of asynchronous MPSTs and do not include features such as delegation or sub-sessions. Let $\mathbb{P}$ be a set of roles and $\mathbb{V}$ be a set of messages. Global types are defined formally with the following grammar:

$$G ::= \sum_{i \in I} p_i \to q_i : m_i.G_i \mid \mu t.G \mid t \mid 0 \tag{1}$$

The terms $p_i \to q_i : m_i$ denote an exchange of a message $m_i \in \mathbb{M}$ between two participants $p_i, q_i \in \mathbb{P}$. The sum operator denotes the choice between some branches. If the branch is just one, we can omit it. The operators μt and t are used for recursion, and 0 denotes termination. We require that recursion must be guarded; more specifically, there must be an interaction between the recursion definition μt and the call of the recursion variable t. Some important restriction are need for the sum operator. A common one is to require that a sender is the same on all sub-brances: $\forall i, j \in I.p_i = p_j$ for every sub-term in $\sum_{i \in I} p_i \to q_i : m_i.G_i$. In the work [2], they define MPSTs' sum operator with the *directed-choice* restriction: upon branching, the sender of the message should be the same and also the receiver, allowing a difference only in the content of the message (additional condition: $\forall i, j \in I.q_i = q_j$). This restriction allow some undesired behavior when we observe the system from an implementability point of view. Therefore, we need a new restriction that can better suits this problem.

Instead, in the thesis [3], there's the definition of *sender-driven choice*: upon branching, the sender can only send different messages to different receivers. Without any restriction, they are called *non-deterministic* global types.

We can have different level of restriction:

- Non-deterministic: no restriction
- Sender-driven choice: for every subterm of $\sum_{i \in I} p_i \to q_i : m_i.G_i$, it holds $\forall i, j \in I, i = j \implies p_i = p_j \vee q_i \neq q_j$.
- Directed-choice: for every subterm of $\sum_{i \in I} p_i \to q_i : m_i.G_i$, it holds $\forall i, j \in I, i = j \implies p_i = p_j \vee q_i = q_j$. This allows to distinguish communication only by the message difference.
- Mixed-choice: for every subterm of $\sum_{i \in I} p_i \to q_i : m_i.G_i$, it holds $\forall i, j \in I, i \neq j \implies p_i \neq p_j \vee q_i \neq q_j \vee m_i \neq m_j$. This condition allows to distinguish all transitions because no sender or receiver or message is equal.

A well-formed global type ensures that all participants can complete their interactions without deadlocks or undefined behavior.

Local Types A local type L represents the communication behavior of an individual participant, derived from the global type. It specifies the actions that a participant can perform within the session and is defined as:

$$L ::= \langle p, S \rangle.T \mid ?(p, U).T \mid \&(p, \{l_i : T_i\}_{i \in I}) \mid \oplus(p, \{l_i : T_i\}_{i \in I}) \mid \mu t.L \mid t \mid \text{end} \tag{2}$$

where:

- ! means the sending of a value,
- ? means the receiving of a message,
- & denotes the selection operation,
- $\oplus$ represents the branching operator,
- $\mu t.L$ represents recursion, allowing repetition of behavior,
- t is a recursion variable,
- `end` denotes the termination of the communication; it can be omitted.

Each participant's local type must be consistent with the global type to ensure correct communication. Recursion is always guarded.

Projection The projection of a global type G onto a participant q ($G \upharpoonright q$) is defined by induction on G:

$$(p \to p' : \langle U \rangle.G') \upharpoonright q = \begin{cases} !\langle p', U \rangle.(G' \upharpoonright q) & \text{if } q = p \\ ?\langle p, U \rangle.(G' \upharpoonright q) & \text{if } q = p' \\ G' \upharpoonright q & \text{otherwise} \end{cases}$$

$$(p \to p' : \{l_i : G_i\}_{i \in I}) \upharpoonright q = \begin{cases} \oplus\langle p', \{l_i : T_i\}_{i \in I} \rangle & \text{if } q = p \\ \&(p, \{l_i : G_i \upharpoonright q\}_{i \in I}) & \text{if } q = p' \\ G_{i_0} \upharpoonright q & \text{if } q \neq p, q \neq p',\ i_0 \in I \\ & \quad \text{and } G_i \upharpoonright q = G_j \upharpoonright q\ \forall i, j \in I \end{cases}$$

$$(\mu t.G) \upharpoonright q = \begin{cases} \mu t.(G \upharpoonright q) & \text{if } G \upharpoonright q \neq t \\ \text{end} & \text{otherwise} \end{cases} \qquad t \upharpoonright q = t \qquad \text{end} \upharpoonright q = \text{end}$$

Well-Formedness

Theorem 1 (Subject Reduction)**.** *Let $\vdash \mathcal{M} : G$. For all $\mathcal{M}'$, if $\mathcal{M} \longrightarrow \mathcal{M}'$, then $\vdash \mathcal{M}' : G'$ for some G' such that $G \Longrightarrow G'$.*

Theorem 2 (Progress)**.** *If $\vdash \mathcal{M} : G$, then either $\mathcal{M} \equiv p \triangleleft \mathbf{0}$ or there is $\mathcal{M}'$ such that $\mathcal{M} \longrightarrow \mathcal{M}'$.*

As a consequence of subject reduction and progress, we get the safety property stating that a typed multiparty session will never get stuck.

Theorem 3 (Type Safety)**.** *If $\vdash \mathcal{M} : G$, then it does not hold stuck($\mathcal{M}$).*

These theorems establish the foundation for reasoning about safe and structured concurrent communications using MPST, ensuring that multi-party protocols are both reliable and verifiable.

2.1.2 The Calculus' Syntax

The process' implementation of the synchronous multiparty session calculus are defined with:

$P ::= \overline{u}[p](y).P$	Multicast request	a, b	Service name
$\mid u[p](y).P$	Accept	x	Value variable
$\mid c!\langle p, e\rangle.P$	Value sending	s	Session name
$\mid c?(p, x).P$	Value reception	p, q	Participant number
$\mid c \oplus \langle p, l\rangle.P$	Selection	X, Y	Process variable
$\mid c\&(p, \{l_i : P_i\}_{i\in I})$	Branching	l	Label
$\mid$ if e then P else Q	Conditional	$u ::= x \mid a$	Identifier
$\mid P \mid Q$	Parallel	$v ::= a \mid$ true $\mid$ false	Value
$\mid 0$	Inaction	$e ::= v \mid x \mid e$ and $e' \mid$ not $e \ldots$	Expression
$\mid (\nu a)P$	Service name hiding	$m ::= (q, p, v) \mid (q, p, s[p']) \mid (q, p, l)$	Message in transit
$\mid$ def D in P	Recursion	$h ::= h \cdot m \mid \emptyset$	Queue
$\mid X\langle e, c\rangle$	Process call	$D ::= X(x, y) = P$	Declaration
$\mid (\nu s)P$	Session hiding		
$\mid s : h$	Message queue		

Table 2. Process syntax and naming conventions.

The evaluation contexts are processes with some holes, and we denote them with:

$$\mathcal{E} ::= [\ ] \parallel P \parallel (\nu a)\mathcal{E} \parallel (\nu s)\mathcal{E} \parallel \text{def } D \text{ in } \mathcal{E} \parallel \mathcal{E}|\mathcal{E}$$

We define a multiparty session as a parallel composition of pairs (denoted by $p \triangleleft P$) of participants and processes:

$$M ::= p \triangleleft P \mid\mid M \mid M \tag{3}$$

with the idea that process P represents participant p and interacts with other processes fulfilling different roles in M, a multiparty session is considered well-formed if all participants are distinct.

2.2 Message Sequence Charts

Definition 2.1 (Communicating automaton)**.** A *communicating automaton* $\mathcal{A}$ is a tuple

$$(L, \mathbb{V}, \mathbb{I}^F, \mathbb{I}^B, \text{Act}, \delta, l_0)$$

where:

- L is a finite set of control states,
- $\mathbb{V}$ is a finite set of messages,
- $\mathbb{I} = \mathbb{I}^F \cup \mathbb{I}^B$ with $\mathbb{I}^B \cap \mathbb{I}^F = \emptyset$ is a finite set of *buffer identifiers*, where $\mathbb{I}^B$ (respectively $\mathbb{I}^F$) is the subset of bag (respectively FIFO) buffer identifiers,
- $\text{Act} \subseteq L \times \{!, ?\} \times \mathbb{V}$ is a finite set of *communicating automaton actions*,
- $\delta \subseteq L \times \text{Act} \times L$ is a finite set of *transitions*, and

- l_0 is the initial control state.

A *communicating automaton action* (denoted a_A) can be:

- a send action: $l!v$, meaning 'send message v in buffer l', or
- a reception: $l?v$, meaning 'receive message v from buffer l'.

For $a_A = l \circ v$ with $\circ \in \{!, ?\}$, buffer $\text{buf}(a_A) = l$ and message $\text{msg}(a_A) = v$.

We mostly consider *deterministic* communicating automata, unless explicitly mentioned otherwise.

Given an alphabet of actions Λ, an *execution* is a word on Λ: it is a finite sequence of actions. An execution is *feasible* in a system if this system is able to exhibit this sequence of actions.

Definition 2.2 (Feasible execution)**.** An execution $e = a_1 \cdot a_2 \cdot \cdots \cdot a_n$ is *feasible* in $\mathfrak{S}$ if there exists a sequence of configurations $\gamma_1, \gamma_2, \ldots, \gamma_n$ such that for all $i \in \{1, \ldots, n\}$, we have:

$$\gamma_{i-1} \xrightarrow{a_i}_{\mathfrak{S}} \gamma_i.$$

The set of all feasible executions of $\mathfrak{S}$ is denoted executions($\mathfrak{S}$). We write:

$$\gamma_0 \xrightarrow{a_1}_{\mathfrak{S}} \gamma_1 \xrightarrow{a_2}_{\mathfrak{S}} \ldots \xrightarrow{a_n}_{\mathfrak{S}} \gamma_n,$$

and by abuse of notation, we write $a \in e = a_0 \cdot \cdots \cdot a_n$ if there exists $i \in \{1, \ldots, n\}$ such that $a = a_i$. If an execution e is feasible in a system $\mathfrak{S}$, we also say that $\mathfrak{S}$ *admits* e.

2.2.1 Communication models

Definition 2.3 (Message Sequence Chart)**.** An MSC over $\mathbb{P} \wedge \mathbb{M}$ is a tuple $M = (\epsilon, \vec{,} \lhd, \lambda)$ where ϵ is a finite (possibly empty) set of events, $\lambda : \epsilon \to \Sigma$ is a labelling function that associates an action to each event, and $\vec{,} \lhd$ are binary relations on ϵ that satisfy the following three conditions. For $p \in \mathbb{P}$, let $\epsilon_p = e \in \epsilon \mid \lambda(e) \in \Sigma_p$ be the set of events that are executed by p.

- The process relation $\vec{\subseteq} \epsilon \times \epsilon$ relates an event to its immediate successor on the same process: $\vec{,} = \bigcup_{p \in \mathbb{P}} \to_p$ for some relations $\to_p \subseteq \epsilon_p \times \epsilon_p$ such that $\to_p$ is the direct successor relation of a total order on ϵ_p.
- The message relation $\lhd \subseteq \epsilon \times \epsilon$ relates pairs of matching send/receive events:
  - For every pair $(e, f) \in \lhd$, there are two processes p, q and a message m such that $\lambda(e) = \text{send}(p, q, m)$ and $\lambda(f) = \text{rec}(p, q, m)$.
  - For all $f \in \epsilon$ with $\lambda(f) = \text{rec}(p, q, m)$, there is exactly one $e \in \epsilon$ such that $e \lhd f$.
  - For all $e \in \epsilon$ such that $\lambda(e) = \text{send}(p, q, m)$, there is at most one $f \in \epsilon$ such that $e \lhd f$.

- The happens-before relation $\leq_{\text{hb}}$, defined by $(\vec{\cup} \lhd)^*$, is a partial order on ϵ.

The projection of a MSC M over a process i is denoted with $M|_i$.

Fully async In the fully asynchronous communication model (`asy`), messages can be received at any time once they have been sent, and send events are non-blocking. It can be modeled as a bag where all messages are stored and retrieved by processes when necessary. It is also referred to as NON-FIFO. The definition is the same as MSC.

Peer-to-peer In the peer-to-peer (`p2p`) communication model, any two messages sent from one process to another are always received in the same order as they are sent. This is usually implemented by processes pairwise connected with FIFO channels. Alternative names are FIFO 1-1 or simply FIFO.

A p2p-MSC is an MSC $M = (E, \rightarrow, \lhd, \lambda)$ where, for any two send events s and s' such that $\lambda(s) \in \text{send}(p, q, _), \lambda(s') in \text{send}(p, q, _)$, and $s \rightarrow^+ s'$, one of the following holds: - either $s, s' \in \text{matched}(M)$ with $s \lhd r$ and $s' \lhd r'$ and $r \rightarrow^+ r'$, - or $s' \in \text{unmatched}(M)$. Note that we cannot have two messages $m1$ and $m2$, both sent by p to q, in that order, such that $m1$ is unmatched and $m2$ is matched; unmatched message $m1$ excludes the reception of any later message.

Causally ordered In the causally ordered (`co`) communication model, messages are delivered to a process according to the causality of their emissions. In other words, if there are two messages $m1$ and $m2$ with the same recipient, such that there exists a causal path from $m1$ to $m2$, then $m1$ must be received before $m2$. Causal ordering was introduced by Lamport with the name "happened before" order.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is causally ordered if, for any two send s and s', such that $\lambda(s) \in \text{send}(_, q, _), \lambda(s') \in \text{send}(_, q, _)$, and $s \leq_{\text{hb}} s'$: - either $s, s' \in \text{matched}(M)$ and $r \rightarrow^* r'$, with r and r' receive events such that $s \lhd r$ and $s' \lhd r'$. - or $s' \in \text{unmatched}(M)$.

Note that in a `co`-MSC we cannot have two send events s and s' addressed to the same process, such that s is unmatched, s' is matched, and $s \leq_{\text{hb}} s'$.

Mailbox In this model, any two messages sent to the same process, regardless of the sender, must be received in the same order as they are sent. If a process receives m_1 before m_2, then m_1 must have been sent before m_2. `mb` coordinates all the senders of a single receiver. This model is also called FIFO $n-1$.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is a `mb`-MSC if it has a linearization $\rightsquigarrow$ where, for any two send events s and s', such that $\lambda(s) \in \text{send}(_, q, _), \lambda(s') \in \text{send}(_, q, _)$, and $s \rightsquigarrow s'$ - either $s, s' \in \text{matched}(M)$ and $r \rightsquigarrow r'$, where $s \lhd r$ and $s' \lhd r'$, - or $s' \in \text{unmatched}(M)$.

FIFO 1-n This model is the dual of `mb`, it coordinates a sender with all the receivers. Any two messages sent by a process must be received in the same order as they are sent. These two messages might be received by different processes and the two receive events might be concurrent.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is a `onen`-MSC if it has a linearization $\rightsquigarrow$ where, for any two send events s and s', such that $\lambda(s) \in \text{send}(p, _, _), \lambda(s') \in \text{send}(p, _, _)$ and $s \rightarrow^+ s'$ (which implies $s \rightsquigarrow s'$) - either $s, s' \in \text{matched}(M)$ and $r \rightsquigarrow r'$, with r and r' receive events such that $s \lhd r$ and $s' \lhd r'$, - or $s' \in \text{unmatched}(M)$.

FIFO n-n In this model, messages are globally ordered and delivered according to their emission order. Any two messages must be received in the same order as they are sent. These two messages might be sent or receives by any process and the two send or receive events might be concurrent. The FIFO `n-n` coordinates all the senders with all the receivers.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is a `nn`-MSC if it has a linearization $\rightsquigarrow$ where, for any two send events s and s', such that $s \rightsquigarrow s'$ - either $s, s' \in \text{matched}(M)$ and $r \rightsquigarrow r'$, with r and r' receive events such that $s \lhd r$ and $s' \lhd r'$, - or $s' \in \text{unmatched}(M)$.

RSC This model imposes the existence of a scheduling such that any send event is immediately followed by its corresponding receive event.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is an `rsc`-MSC if it has no unmatched send events and there is a linearization $\rightsquigarrow$ where any matched send event is immediately followed by its respective receive event.

3 State of the art

In this section realizability and implementability will be used as the same problem. Informally, an MSC can be defined realizable if a distributed implementation exists that exactly reproduces the graph's behaviors.

3.1 Realizability by Alur

3.1.1 Definitions

Definition 3.1 (Weakly-implies)**.** A set L of MSCs weakly implies an MSC M iff for all $1 \leq i \leq n$, there exists an MSC $M_i \in L$ such that $M|_i = M_i|_i$.

Definition 3.2. Weakly realizable The weak closure L^w of a set L of MSCs contains all the MSCs L weakly implies. The set L is weakly realizable iff $L = L^w$. The MSC-graph G is said to be weakly realizable if the set $L(G)$ of MSCs is. Thus, a weakly realizable graph already contains all the implied scenarios.

Definition 3.3 (Deadlock-free)**.** Consider a set A_i of concurrent automata and the product $A = \prod_i A_i$ A state q of the product A is said to be a deadlock state if no accepting state of A is reachable from q. The product A is said to be deadlock-free if no state reachable from its initial state is a deadlock state.

Definition 3.4 (Safely realizable)**.** A set L of MSCs is said to be safely realizable if $L = L \prod A_i$ for some $\langle A_i | 1 \leq i \leq n \rangle$ such that $\prod A_i$ is deadlock-free.

3.1.2 Realizability problem

In [1], they prove many interesting results within the scope of MSC graphs. They first examine the realizability of bounded MSC-graphs. Two types are considered: *weak* (allowing deadlocks) and *safe* (deadlock-free). While weak realizability is `coNP`-complete and safe realizability solvable in polynomial time for finite MSC sets, they show that for bounded MSC-graphs, weak realizability becomes undecidable, and safe realizability lies in `EXPSPACE`.

Theorem 4. *Given a bounded MSC graph G, checking if G is weakly realizable is undecidable.*

Proof. The proof is a reduction from the post correspondence problem (PCP). In the paper [1] they further reduce the problem to a relaxed version. First, they reduce PCP to OneSidedPCP and then to Relaxed PCP (RPCP). We skip this part as is not meaningful for the proof. We can state the RPCP to be the following problem: given $\{(v_1, w_1), (v_2, w_2), ..., (v_r, w_r)\}$, determinate whether there are indices $i_1, ..., i_m$ such that $x_{i_1}...x_{i_n} = y_{i_1}...y_{i_m}$, where $x_{i_j}, y_{i_j} \in \{v_{i_j}, w_{i_j}\}$, for some index i_l $x_{i_l} \neq y_{i_l}$, and for all $j \leq m, y_{i_1}...y_{i_j}$ is a prefix of $x_{i_1}...x_{i_j}$. The relaxed problem essentially says that must have the left side of the equivalence that grows more than the right side, we must have also some tuple that are different, and that we can choose whatever side of tuple for the composition of the string. Now we reduce RPCP to weak realizability.

Encodig: Given a finite set L of MSCs, let L^* denote the MSC graph that consists of the *complete* graph with $|L|$ vertices, one per MSC in the set L, dummy initial and terminal vertices v_I, v_T with empty MSCs, and edges from v_I to all vertices of L and from those to v_T. Thus, an MSC of this graph is simply a concatenation of MSCs from the set L.

In the sequel, we say that a process p synchronously sends a message m to process q, if p sends m to q immediately followed by q sending the message m back to p. In figures, such messages will be depicted by double arrows.

Given an instance $\Delta = \{(v_1, w_1), \ldots, (v_m, w_m)\}$ of RPCP, we build a set L of MSCs over 4 processes as follows. For each pair (v_i, w_i) we build two MSCs M_i^0 and M_i^1, which are depicted in Fig. 3. Thus, in M_i^0, process 1 sends synchronously $(i, 0)$ to process 2, then sends the index i to process 4, and then process 4 sends synchronously $(i, 0)$ to process 3. After that, process 2 synchronously sends the sequence of characters of v_i to process 3 (note we assume c is the length of v_i and d the length of w_i in the figure), M_i^1 is similar. Observe that the communication graph of each of these MSCs is strongly connected and involves all the processes, and hence, the MSC graph L^* is bounded.

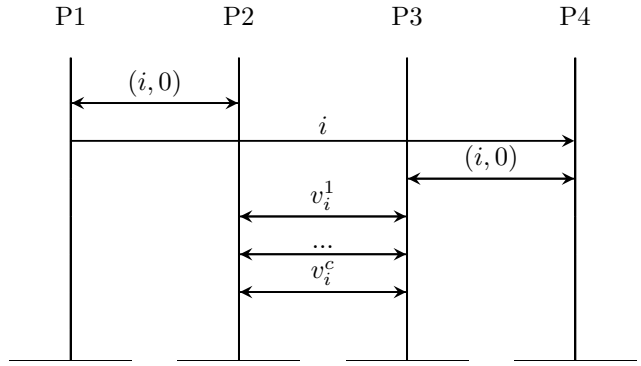

Figure 1: The M_i^0 MSC. For a string u, let u_l denote the l'th character of the string.

Claim 1: $\Delta \in$ RPCP iff L^* is not weakly realizable.

Proof. For the "only if" direction, suppose $R = (i_1, a_1, b_1, i_2, a_2, b_2, \ldots, i_m, a_m, b_m)$ are the indices for a solution to $\mathcal{B}$, and the bits a_j and b_j indicate which string (v_{i_j} or w_{i_j}) is chosen to go into the two (left and right) long strings.

Consider the new MSCs M and M' obtained from the sequences $M = M_{i_1}^{a_1} \cdots M_{i_m}^{a_m}$ and $M' = M_{i_1}^{b_1} \cdots M_{i_m}^{b_m}$. Executions of both of these (sequences of) MSCs must exist in any realization of L^*. We then look at the projections $M|_1$, $M|_2$, $M|_3$, and $M|_4$ of M, and $M'|_1$, $M'|_2$, $M'|_3$, $M'|_4$ of M' onto the 4 processes. Now consider an MSC M'' formed from $M'|_1$, $M'|_2$, $M|_3$, and $M|_4$. The claim is that the combined MSC M'' is weakly implied by L^*. By definition, the only thing to establish is that M'' is indeed an MSC, in the sense that it is acyclic, well-formed, and complete.

The only new situation in terms of communication in M'' is the communication between P_1 and P_4, and between P_2 and P_3. But the communication between P_1 and P_4 is consistent in $M'|_1$ and $M|_4$ (i.e., the sequence of messages sent from P_1 to P_4 in $M'|_1$ is equal to the sequence of messages received in $M|_4$), and the communication between P_2 and P_3 is consistent in $M'|_2$ and $M|_3$ because R is a solution to the RPCP.

Furthermore, the acyclicity of M'' follows from the property of the solution that the string formed by the first j words on processes 1 and 2 is always a prefix of the string formed by the first j words on processes 3 and 4. Consequently, each message from P_1 to P_4 is sent before it needs to be received.

But note that M'' cannot itself be in L^* because there must be some index i_j where $a_j \neq b_j$, and no MSC exists in L where, after process 1 announces the index, what process 2 sends is not identical to what process 3 receives.

Now, for the "if" direction, suppose there is some MSC $M^{@}$ which exists in any realization of L^*, but is not in L^* itself. We want to derive a solution to $\mathcal{B}$

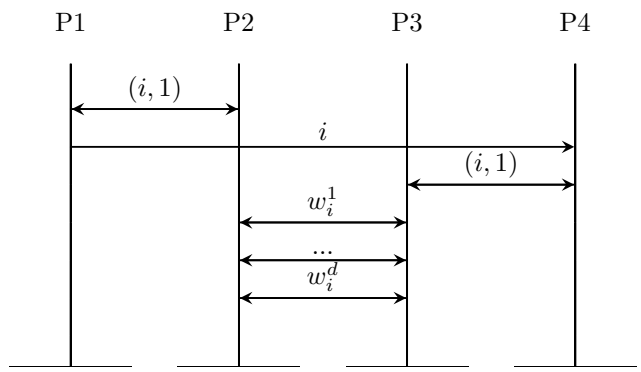

Figure 2: The M_i^1 MSC. For a string u, let u_l denote the l'th character of the string.

from $M^{@}$.

First, it is clear that the projection $M^{@}|_1$ must consist of a sequence of pairs of messages (the first of each pair acknowledged), sent from process 1 to processes 2 and 4, respectively, with messages (i, b) and i.

Likewise, it is clear that, in order for process 2 to receive those messages, $M^{@}|_2$ must consist of a sequence of receipts of (i, b) pairs, and after each (i, b), either v_i or w_i is sent to process 3, based on whether $b = 0$ or $b = 1$, before the next index pair is received.

Likewise, $M^{@}|_4$ consists of a sequence of receipts of index i from process 1, followed by sending of $(i, 0)$ or $(i, 1)$ to process 3, and $M^{@}|_3$ consists of a sequence of receipt of $(i, 0)$ or $(i, 1)$ followed by receipt of v_i or w_i, respectively.

Now, since $M^{@}$ is not in L^*, for some index i the choice of v_i or w_i must differ on process 2 and process 3. (Note, we are assuming that the buffers between processes are FIFO.)

Furthermore, because of the precedences, the prefix formed by the first j words on process 2 must precede the $(j + 1)$-th message from process 1 to process 4, which in turn precedes the $(j+1)$-th message from 4 to 3, and hence the $(j+1)$-th word on process 3. That is, the string formed by the first j words on process 2 is a prefix of the string formed by the first j words on process 3. Therefore, we can readily build a solution for $\mathcal{B}$ from $M^{@}$.

□

3.2 Implementability by Stutz

Part taken from [3]. To start with, let P be a finite set of participants and V be a finite set of messages and Γ the set of all send and receive events. Γ_p is the set of all send and receive events regarding a participant $p \in P$.

3.2.1 Definitions

Definition 3.5. CSM A *communicating state machine* (CSM) is $\mathsf{A} = \{\!\{A_p\}\!\}_{p \in P}$ over P and V if A_p is a finite state machine over Γ_p for every $p \in P$, denoted by $(Q_p, \Gamma_p, \delta_p, q_{0,p}, F_p)$. Let $\prod_{p \in P} Q_p$ denote the set of global states and $\texttt{Chan} = \{(p,q) | p, q \in P, p \neq q\}$ denote the set of channels. A configuration of A is a pair $(\overrightarrow{q}, \mathcal{E})$, where $\overrightarrow{q}$ is a vector of sates, one for each participant, and $\mathcal{E} : \text{Chan} \to V^*$ is a mapping from each channel to a sequence of messages. We may write $\mathcal{E}(p,q)$ for $\mathcal{E}((p,q))$. We use $\overrightarrow{q}_p$ to denote the state of p in $\overrightarrow{q}$. The relation, denoted $\to$, is defined as follows:

- $(\overrightarrow{q}, \mathcal{E}) \xrightarrow{p \triangleright q!m} (\overrightarrow{q}', \mathcal{E}')$ if $(\overrightarrow{q_p}, p \triangleright q!m, \overrightarrow{q_p}') \in \delta_p, \overrightarrow{q_r} = \overrightarrow{q_r}'$ for every participant $r \neq p$, $\mathcal{E}'((p,q)) = \mathcal{E}((p,q)) \cdot m$ and $\mathcal{E}'(c) = \mathcal{E}(c)$ for every other channel $c \in \text{Chan}$.
- $(\overrightarrow{q}, \mathcal{E}) \xrightarrow{q \triangleleft p?m} (\overrightarrow{q}', \mathcal{E}')$ if $(\overrightarrow{q_p}, q \triangleleft p?m, \overrightarrow{q_p}') \in \delta_p, \overrightarrow{q_r} = \overrightarrow{q_r}'$ for every participant $r \neq p$, $\mathcal{E}((p,q)) = m \cdot \mathcal{E}'((p,q))$ and $\mathcal{E}'(c) = \mathcal{E}(c)$ for every other channel $c \in \text{Chan}$.
- $(\overrightarrow{q}, \mathcal{E}) \xrightarrow{\epsilon} (\overrightarrow{q}', \mathcal{E})$ if $(\overrightarrow{q_p}, \epsilon, \overrightarrow{q_p}') \in \delta_p$ for some participant p, and $\overrightarrow{q_q} = \overrightarrow{q_q}'$ for every participant $q \neq p$.

Informally, a configuration $(\overrightarrow{q}, \mathcal{E})$ is a deadlock if it is not final and has no outgoing transitions while it is reachable if it occurs on some run of A. A CSM is deadlock-free if no reachable configuration is a deadlock. Γ^ω denotes the infinite set of events.

Definition 3.6 (Implementability problem)**.** A language $L \subseteq \Gamma^\omega$ is said to be implementable if there exists a CSM $\{\!\{A_p\}\!\}_{p \in P}$ such that

- deadlock freedom: $\{\!\{A_p\}\!\}_{p \in P}$ is deadlock-free, and
- protocol fidelity: L is the language of $\{\!\{A_p\}\!\}_{p \in P}$.

References

[1] Rajeev Alur, Kousha Etessami, and Mihalis Yannakakis. Realizability and verification of msc graphs. *Theoretical Computer Science*, 331(1):97–114, 2005.

[2] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 273–284, 2008.

[3] Felix Stutz. *Implementability of Asynchronous Communication Protocols–The Power of Choice*. PhD thesis, Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau, 2024.